RAPPORT DE PROJET — DÉVELOPPEMENT LOGICIEL

Finance Paiements

Portefeuille électronique et paiements numériques

Analyse, conception, réalisation, tests, audit de sécurité et déploiement d'une plateforme de portefeuille électronique

Réalisé par

LUMU BAMANA MANASSE

Début du projet	**août 2026**
Dernière mise à jour	**22 octobre 2026**
Version	**1.1.0**
Modèle de cycle de vie	**Prototypage évolutif**
Méthode	**Agile — Scrum (5 prototypes, 12 sprints)**
Technologies	**Python 3.12 · Django 6 · Django REST Framework**
	PostgreSQL 16 · Docker · GitHub Actions
Code source	`github.com/MediJr/finance-paiements`

169 tests automatisés · 86 % de couverture · 11 défauts corrigés lors de l'audit · Licence MIT

Résumé

Finance Paiements est une plateforme de **portefeuille électronique**. Chaque client dispose d'un portefeuille en euros qu'il alimente par carte bancaire, avec lequel il vire de l'argent à ses proches, paie des commerçants et reçoit des remboursements. Le personnel supervise l'activité, fixe les plafonds, peut suspendre les opérations et consulte journaux, statistiques et exports. Les fonctions sont accessibles par une interface web et par une API REST.

Le logiciel a été développé selon le **modèle par prototypage évolutif** : cinq prototypes fonctionnels successifs, chacun enrichissant le précédent. À l'intérieur de chaque prototype, le travail a été organisé selon la méthode **agile Scrum**, en douze sprints (0 à 11), l'auteur tenant lui-même le rôle du client lors de l'évaluation de chaque prototype.

Ce rapport retrace toutes les phases du cycle de vie du développement logiciel : l'**analyse des besoins** (15 exigences fonctionnelles, 13 exigences non fonctionnelles, 20 récits utilisateurs), la **conception** (architecture en couches, module unique de mouvements de fonds, modèle de données), la **réalisation** sprint par sprint, les **tests** (169 tests automatisés, 86 % de couverture, matrice de traçabilité), l'**audit de sécurité** mené par l'auteur sur son propre code (11 défauts identifiés et corrigés), le **déploiement** et la **maintenance**.

Chiffres clés

11 modèles de données	**31** routes d'API REST	**35** pages web	**169** tests automatisés
86 % de couverture	**12** sprints	**11** défauts corrigés	**0** alerte de configuration

Table des matières

Résumé **i**

1 Présentation du logiciel **1**

1.1 Contexte . . . 1

1.2 À quoi sert le logiciel . . . 1

1.3 Acteurs . . . 1

1.4 Aperçu . . . 2

1.5 Périmètre . . . 2

2 Démarche : modèle de cycle de vie et méthode **3**

2.1 Modèle : le prototypage évolutif . . . 3

2.2 Méthode : Agile Scrum adapté . . . 3

2.3 Phases du cycle de vie et livrables . . . 5

3 Analyse des besoins **6**

3.1 Exigences fonctionnelles . . . 6

3.2 Exigences non fonctionnelles . . . 7

3.3 Récits utilisateurs . . . 7

3.4 Contraintes . . . 8

4 Conception **9**

4.1 Architecture en couches . . . 9

4.2 Modèle de données . . . 10

4.3 Flux d'un virement . . . 11

5 Réalisation **12**

5.1 Environnement technique . . . 12

5.2 Déroulé des prototypes et des sprints . . . 12

5.3 Évolutions issues des évaluations . . . 13

5.4 Parcours réalisé . . . 13

6 Tests et validation **15**

6.1 Stratégie . . . 15

6.2 Résultats . . . 15

6.3 Matrice de traçabilité . . . 16

6.4 Intégration continue 17

7 Audit de sécurité **18**

7.1 Démarche 18

7.2 Modèle de menace 18

7.3 Défauts identifiés et corrigés 18

7.4 Exemple détaillé : le double débit (F4) 19

8 Déploiement **20**

8.1 Environnements 20

8.2 Mise en production 20

9 Maintenance et évolutions **21**

9.1 Gestion de version 21

9.2 Limites connues 21

9.3 Évolutions envisagées 21

10 Conclusion **22**

A Annexes **23**

A.1 Comptes et cartes de démonstration 23

A.2 Organisation du dépôt 23

Table des figures

1.1 Interface client : solde, actions rapides et alimentation 2

2.1 Prototypage évolutif : cinq prototypes successifs, chacun évalué avant le suivant . . 3

4.1 Architecture en couches 9
4.2 Principales entités du modèle de données (11 modèles) 10
4.3 Séquence d'un virement : tout échec annule la transaction entière 11

5.1 Virement et pilotage 13
5.2 Paramètres : plafonds par opération, journalier et d'alimentation, maintenance . . 14

6.1 Pipeline d'intégration continue (deux jobs parallèles) 17

Liste des tableaux

1.1 Acteurs du système 1

2.1 Justification du choix du modèle 3
2.2 Mise en œuvre de Scrum 4
2.3 Phases du cycle de vie et livrables correspondants dans le dépôt 5

3.1 Exigences fonctionnelles 6
3.2 Exigences non fonctionnelles 7
3.3 Extrait du product backlog 8

4.1 Modules (applications Django) 9

5.1 Technologies utilisées 12
5.2 Réalisation sprint par sprint 12
5.3 Retours d'évaluation et évolutions apportées 13

6.1 Niveaux de test 15
6.2 Répartition des tests par fichier 15
6.3 Matrice de traçabilité exigences / tests (extrait) 16

7.1 Actifs, menaces et attaquants 18
7.2 Résultats de l'audit de sécurité (sprint 11) 18

8.1 Configurations par environnement 20

A.1 Comptes de démonstration (mode développement uniquement) 23
A.2 Cartes de test de la passerelle simulée 23
A.3 Structure du dépôt 23

CHAPITRE 1

Présentation du logiciel

1.1 Contexte

Les applications de paiement entre particuliers et les néobanques ont popularisé le portefeuille électronique : un solde en ligne que l'on approvisionne une fois, puis que l'on utilise pour envoyer ou recevoir de l'argent instantanément. Un établissement fictif souhaite proposer ce service à ses clients, avec un haut niveau d'exigence sur l'intégrité des fonds, la confidentialité et la traçabilité.

1.2 À quoi sert le logiciel

Pour le client ouvrir un compte, alimenter son portefeuille par carte, gérer ses bénéficiaires, effectuer des virements et des paiements, rembourser un paiement reçu, consulter son historique.

Pour l'établissement superviser utilisateurs, portefeuilles et opérations, activer ou désactiver un compte, fixer les plafonds (par opération, journalier, d'alimentation), suspendre les opérations (mode maintenance).

Pour la direction suivre l'activité par un tableau de bord, des statistiques et des rapports ; exporter les données en CSV ou JSON.

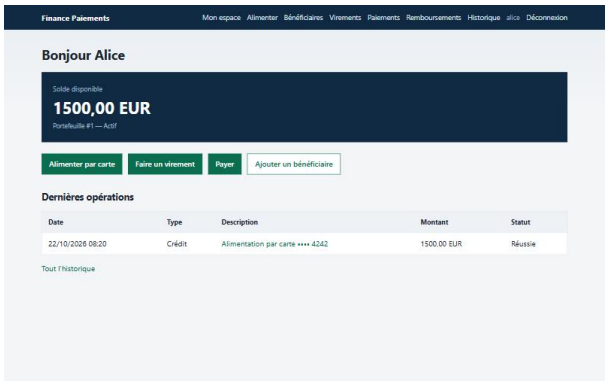
Pour l'audit disposer d'un journal immuable des événements sensibles (connexions, alimentations, virements, actions d'administration).

1.3 Acteurs

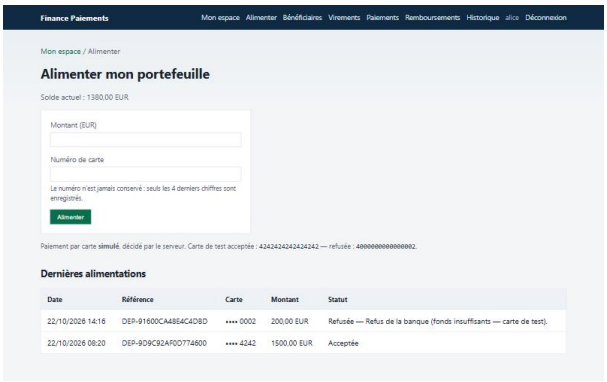
Table 1.1 – Acteurs du système

Acteur	Rôle
Visiteur	Crée un compte et se connecte.
Client	Possède un portefeuille ; alimente, vire, paie, rembourse, consulte.
Administrateur	Membre du personnel ; supervise, paramètre, active ou désactive des comptes, consulte journaux et exports.

1.4 Aperçu



(a) Espace client



(b) Alimentation par carte

Figure 1.1 – Interface client : solde, actions rapides et alimentation

1.5 Périmètre

Le traitement de la carte bancaire est **simulé** : une passerelle côté serveur reproduit le comportement d'un prestataire (validation du numéro, autorisation, refus, masquage). Sont hors périmètre : un prestataire de paiement réel, les virements vers un compte bancaire externe, le multi-devise avec conversion, la vérification d'identité (KYC) et les notifications par e-mail ou SMS.

CHAPITRE 2

Démarche : modèle de cycle de vie et méthode

2.1 Modèle : le prototypage évolutif

Le projet suit le **modèle par prototypage évolutif**. Le logiciel est construit sous forme de prototypes successifs ; chaque prototype est *fonctionnel*, *testé* et *utilisable*. Il n'est pas jeté après évaluation mais **enrichi** par le suivant, jusqu'au produit final.

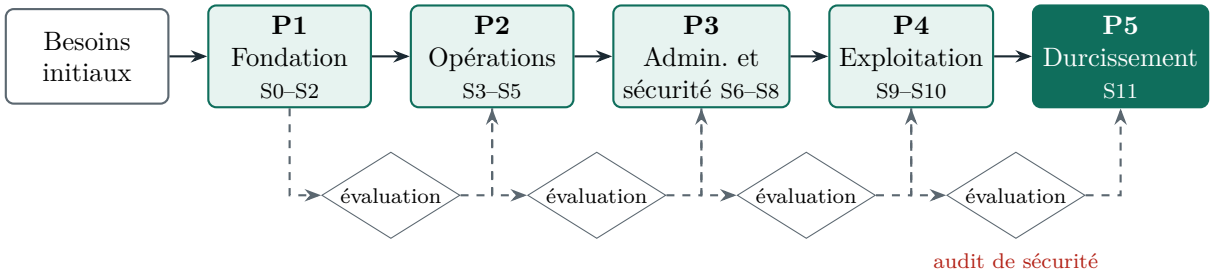


Figure 2.1 – Prototypage évolutif : cinq prototypes successifs, chacun évalué avant le suivant

Table 2.1 – Justification du choix du modèle

Critère	Justification
Besoins qui se précisent en cours de route	Un portefeuille électronique se comprend en le manipulant : l'impossibilité d'approvisionner un portefeuille n'est apparue qu'en jouant le parcours complet du prototype 4.
Risques maîtrisés tôt	Le cœur sensible — les mouvements de fonds atomiques — est construit et testé dès le prototype 2, avant toute interface d'administration.
Sécurité par couches	Le prototype 3 est consacré à la *Security by Design*, le prototype 5 à l'audit et au durcissement.
Développeur unique	Pas de lourdeur documentaire initiale : la documentation grandit avec le produit.

2.2 Méthode : Agile Scrum adapté

À l'intérieur de chaque prototype, le travail est découpé en **sprints** courts selon la méthode **Scrum**. Le projet étant mené par une seule personne, les rôles et les cérémonies ont été adaptés : l'auteur est à la fois Product Owner, équipe de développement et client lors des évaluations.

Table 2.2 – Mise en œuvre de Scrum

Élément Scrum	Mise en œuvre dans le projet
Product Owner	L'auteur définit et priorise le backlog (méthode MoSCoW).
Équipe de développement	L'auteur.
Client	L'auteur, dans le rôle d'utilisateur final : chaque prototype est évalué en le manipulant comme client puis comme administrateur.
Product backlog	20 récits utilisateurs reliés aux exigences (`docs/backlog.md`).
Sprint planning	Objectif de sprint et récits sélectionnés, en tête de chaque rapport de sprint.
Incrément	Code, migrations, tests et documentation livrés ensemble.
Sprint review	Démonstration du parcours et validation de l'objectif.
Rétrospective	Section « Limites / suite » des rapports de sprint, reprise au sprint suivant.

Definition of Done

Un récit utilisateur est considéré comme **terminé** lorsque :

1. le code est écrit dans la couche adaptée (service métier, puis vue ou API) ;
2. toute modification de solde passe par le module de mouvements de fonds ;
3. les migrations sont générées et `makemigrations -check` ne détecte rien ;
4. des tests automatisés couvrent le cas nominal *et* les cas d'erreur ;
5. la suite complète de tests passe ;
6. les actions sensibles sont journalisées (à partir du sprint 9) ;
7. la documentation du module et le rapport de sprint sont à jour.

2.3 Phases du cycle de vie et livrables

Table 2.3 – Phases du cycle de vie et livrables correspondants dans le dépôt

Phase	Contenu	Livrables
1. Analyse des besoins	Contexte, acteurs, exigences, récits utilisateurs	`cahier-des-charges.md`, `backlog.md`
2. Conception	Architecture, modèle de données, flux de fonds	`architecture.md`
3. Réalisation	Développement incrémental par sprints	Code, `livrables/Sprint-XX`
4. Tests	Unitaires, intégration, sécurité, CI	`tests/`, `tests.md`, CI
5. Audit et validation	Audit technique et de sécurité, remédiation	`securite.md`, `livrables/Audit`
6. Déploiement	Docker, production	`docker/`, `installation.md`
7. Maintenance	Historique, versions, évolutions	`CHANGELOG.md`, `versions.md`

CHAPITRE 3

Analyse des besoins

L'analyse des besoins formalise ce que le logiciel doit faire (*exigences fonctionnelles*) et les qualités qu'il doit présenter (*exigences non fonctionnelles*). Chaque exigence est numérotée afin d'être reliée au code puis aux tests qui la vérifient (section 6.3). La priorité suit la méthode **MoSCoW** : **M** (*Must*, indispensable), **S** (*Should*, importante), **C** (*Could*, souhaitable).

3.1 Exigences fonctionnelles

Table 3.1 – Exigences fonctionnelles

ID	Exigence	Prio.	Sprint
Comptes et portefeuilles			
EF-01	Un visiteur crée un compte ; un portefeuille en euros à solde nul est ouvert automatiquement.	M	1, 11
EF-02	Un client se connecte et se déconnecte depuis un espace dédié, distinct du site d'administration.	M	11
EF-03	Un client consulte son solde, son profil et ses dernières opérations.	M	1, 11
EF-04	Un client alimente son portefeuille par carte ; la passerelle accepte ou refuse.	M	11
Opérations			
EF-05	Chaque opération est inscrite dans un historique consultable et filtrable.	M	2
EF-06	Un client gère ses bénéficiaires (ajout par adresse e-mail, alias, désactivation).	M	3, 11
EF-07	Un client vire de l'argent à un bénéficiaire actif ; débit et crédit sont atomiques.	M	4
EF-08	Un client paie un bénéficiaire, qui peut le rembourser totalement ou partiellement.	M	5
EF-09	Une demande envoyée deux fois (double clic, réseau) n'est exécutée qu'une fois.	M	11
Administration et pilotage			
EF-10	Supervision des utilisateurs, portefeuilles et opérations ; activation et désactivation des comptes.	M	6
EF-11	Paramètres : plafonds par opération, journalier et d'alimentation, devise, durée de session, mode maintenance.	M	6, 7, 11
EF-12	Les actions d'administration sont journalisées.	M	7
EF-13	Tableau de bord, statistiques et rapports filtrables.	S	8
EF-14	Journal d'événements centralisé et consultable.	S	9

ID	Exigence	Prio.	Sprint
EF-15	Export CSV / JSON de 8 jeux de données.	C	10

3.2 Exigences non fonctionnelles

Table 3.2 – Exigences non fonctionnelles

ID	Catégorie	Exigence	Mise en œuvre
ENF-01	Sécurité	Mots de passe hachés et validés ; message d'échec identique que le compte existe ou non.	Validateurs Django, PBKDF2
ENF-02	Sécurité	Un client n'accède qu'à ses données ; administration réservée au personnel.	Filtrage dans les services
ENF-03	Sécurité	Aucun annuaire de clients exposé.	Ajout par e-mail exact
ENF-04	Sécurité	Blocage après 5 échecs de connexion (compte) ou 20 (IP).	`auth_service` + cache
ENF-05	Sécurité	CSRF, XSS, clickjacking, redirections ouvertes, injection CSV.	Middleware, `commun/`
ENF-06	Sécurité	Aucune donnée de carte complète conservée.	`carte_masquee`
ENF-07	Intégrité	Ni solde négatif ni double débit.	Transactions, verrous, idempotence
ENF-08	Intégrité	Montants décimaux à 2 décimales, sans arrondi silencieux.	`parse_montant`
ENF-09	Conformité	Plafond par opération et plafond journalier cumulé.	`securite_service`
ENF-10	Traçabilité	Événements immuables pour toute action sensible.	`journal_service`
ENF-11	Qualité	Couverture $\geq 80\,\%$, tests à chaque push.	pytest-cov, GitHub Actions
ENF-12	Déploiement	Démarrage refusé sans clé secrète ; HTTPS en production.	`settings/production.py`
ENF-13	Maintenabilité	Un seul module modifie les soldes ; logique dans les services.	`mouvement_service`

3.3 Récits utilisateurs

Les exigences ont été traduites en **20 récits utilisateurs** au format « En tant que… je veux… afin de… ». Quelques exemples :

Table 3.3 – Extrait du product backlog

ID	Récit utilisateur	Exigence	Sprint
US-05	En tant que client, je veux virer de l'argent à un bénéficiaire, sans risque de débit sans crédit.	EF-07, ENF-07	4
US-08	En tant qu'administrateur, je veux fixer des plafonds et suspendre les opérations en cas d'incident.	EF-11	6, 7
US-09	En tant que client, je veux être certain qu'aucun autre client ne voit mes données.	ENF-02	7
US-15	En tant que client, je veux alimenter mon portefeuille par carte afin de pouvoir l'utiliser.	EF-04	11
US-16	En tant que client, je veux qu'un double clic ne me débite jamais deux fois.	EF-09	11
US-17	En tant que client, je veux que personne ne puisse obtenir la liste des clients.	ENF-03	11

3.4 Contraintes

— Technologies : Python 3.12, Django 6, Django REST Framework, PostgreSQL 16.

— Projet mené par une seule personne, qui tient aussi le rôle de Product Owner.

— Données de démonstration uniquement ; aucune transaction bancaire réelle.

CHAPITRE 4

Conception

4.1 Architecture en couches

L'application est un monolithe Django **modulaire**. L'interface web et l'API REST partagent une même **couche de services métier**. Un module unique, `mouvement_service`, est autorisé à débiter et à créditer les portefeuilles.

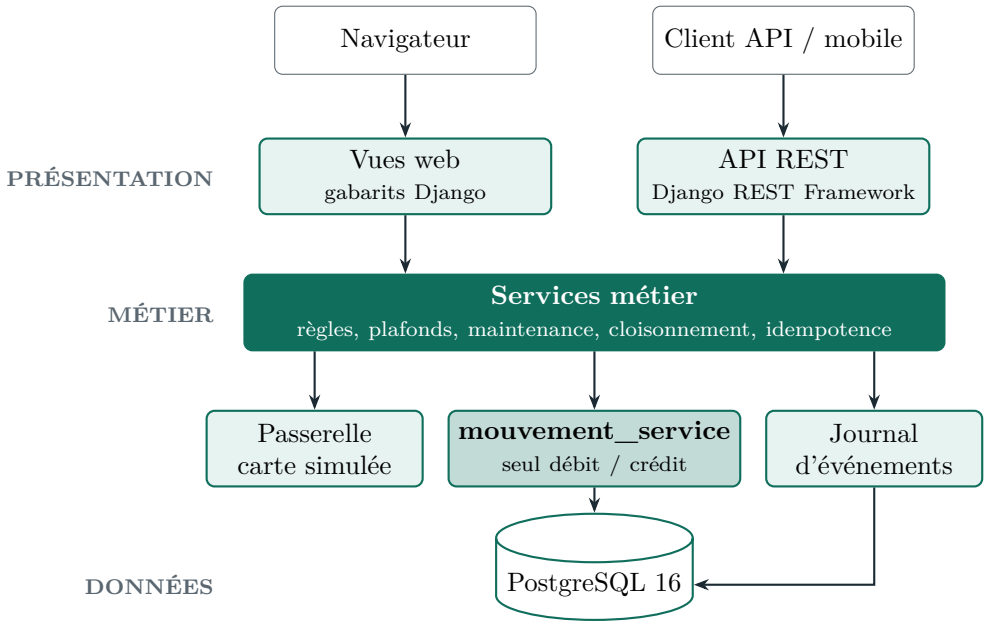


Figure 4.1 – Architecture en couches

Table 4.1 – Modules (applications Django)

Module	Rôle
`accounts`	Utilisateurs métier, inscription, connexion protégée, espace client
`wallets`	Portefeuilles, mouvements de fonds, alimentation par carte
`historique`	Journal des opérations de chaque portefeuille
`beneficiaires`	Destinataires autorisés (ajout par e-mail)
`virements`	Virements entre portefeuilles
`paiements`	Paiements et remboursements
`administration`	Paramètres, supervision, sécurité, pilotage, exports
`observabilite`	Journal d'événements immuable, erreurs HTTP 500
`api`	Routes REST (client et administration)
`commun`	Adresse IP fiable, redirections sûres, clé d'idempotence

4.2 Modèle de données

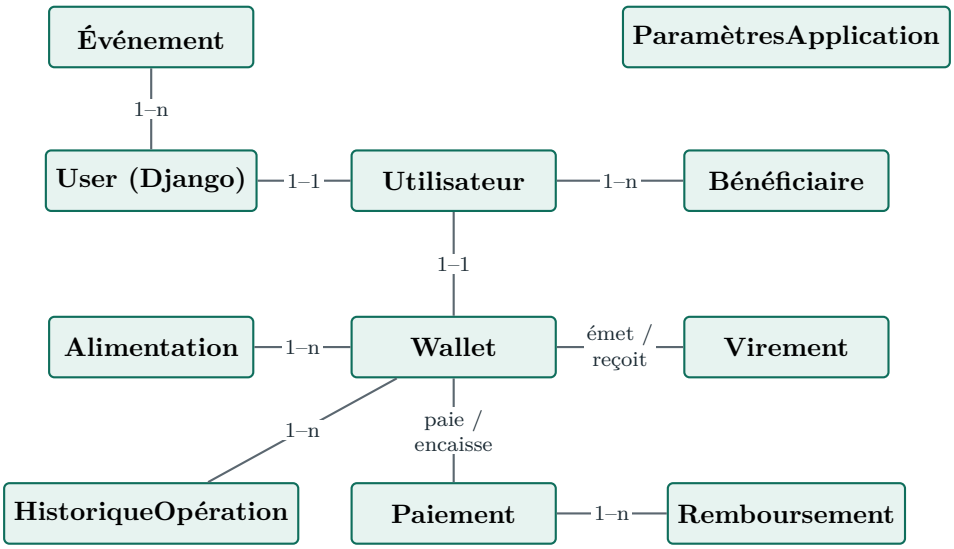


Figure 4.2 – Principales entités du modèle de données (11 modèles)

Choix de conception notables.

— **Solde jamais négatif** : le solde est contrôlé *après* le verrouillage des deux portefeuilles, dans la même transaction que l'écriture.

— **Verrous dans un ordre stable** (identifiants triés) : deux virements croisés ne peuvent pas s'interbloquer.

— **Idempotence** : `Virement`, `Paiement` et `Alimentation` portent une clé unique par portefeuille émetteur, garantie par une contrainte en base.

— **Historique double** : chaque transfert écrit une ligne de sortie chez l'émetteur et une ligne d'entrée chez le destinataire.

— **Paramètres singleton** : une seule ligne de paramètres globaux, éditable par l'administration et journalisée.

4.3 Flux d'un virement

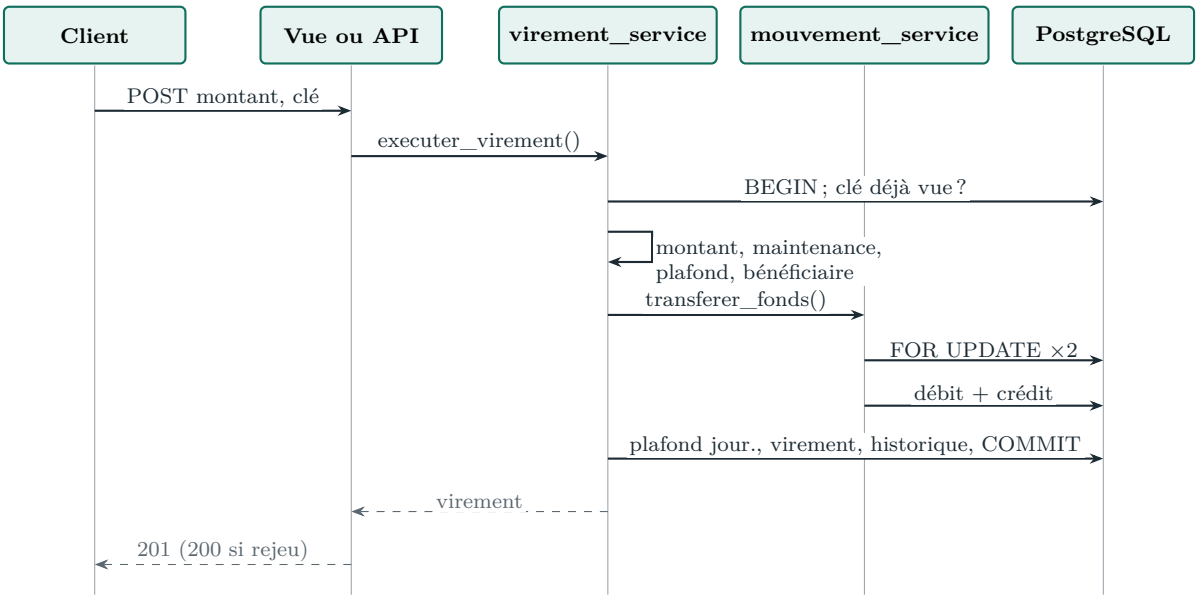


Figure 4.3 – Séquence d'un virement : tout échec annule la transaction entière

Si *n'importe quelle* vérification échoue, la transaction est annulée : aucun débit sans crédit, aucun virement sans historique. Ce comportement est couvert par un test qui provoque volontairement une erreur après le débit.

CHAPITRE 5

Réalisation

5.1 Environnement technique

Table 5.1 – Technologies utilisées

Technologie	Version	Rôle
Python	3.12	Langage
Django	6.0.7	Framework web
Django REST Framework	3.17.1	API REST
PostgreSQL	16	Base de données
pytest, pytest-django, pytest-cov	9.1 / 4.12 / 6.2	Tests et couverture
Docker, Docker Compose	—	Conteneurisation
Gunicorn, WhiteNoise	23.0 / 6.9	Serveur d'application, fichiers statiques
GitHub Actions	—	Intégration continue

5.2 Déroulé des prototypes et des sprints

Table 5.2 – Réalisation sprint par sprint

Prototype	Sprint	Objectif	Résultat
P1 Fondation	0	Initialiser le projet	Django, DRF, PostgreSQL, Docker, tests de configuration
	1	Utilisateurs et portefeuilles	Portefeuille créé automatiquement, solde initial nul
	2	Historique	Journal d'opérations par portefeuille, filtres
P2 Opérations	3	Bénéficiaires	Destinataires autorisés, désactivation logique
	4	Virements	Débit et crédit atomiques, verrous, historique automatique
	5	Paiements, remboursements	Remboursement total ou partiel, factorisation des mouvements
P3 Administration et sécurité	6	Administration	Paramètres, supervision, activation des comptes

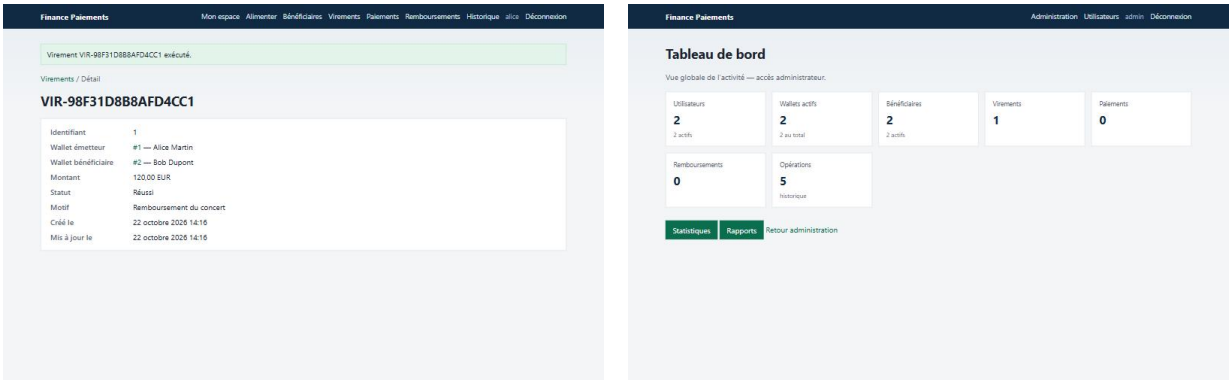
Prototype	Sprint	Objectif	Résultat
	7	Security by Design	Cloisonnement, plafonds et maintenance appliqués, journal d'administration
	8	Pilotage	Tableau de bord, statistiques, rapports
P4 Exploitation	9	Observabilité	Journal d'événements immuable, erreurs 500 tracées
	10	Exports	CSV / JSON, 8 jeux de données
P5 Durcissement	11	Audit et remédiation	11 défauts corrigés, espace client, alimentation, 48 tests, CI, production

5.3 Évolutions issues des évaluations

Table 5.3 – Retours d'évaluation et évolutions apportées

Constat	Proto.	Évolution
L'API exposait encore les profils et portefeuilles d'autres clients	3	Cloisonnement par propriétaire (sprint 7)
Plafonds et maintenance configurables mais non appliqués	3	Application réelle dans les services (sprint 7)
Un client ne pouvait ni se connecter ni approvisionner son portefeuille	4	Espace client et alimentation par carte (sprint 11)
Double débit possible ; plafond par opération contournable	4	Idempotence et plafond journalier (sprint 11)

5.4 Parcours réalisé



(a) Virement exécuté (b) Tableau de bord administrateur

Figure 5.1 – Virement et pilotage

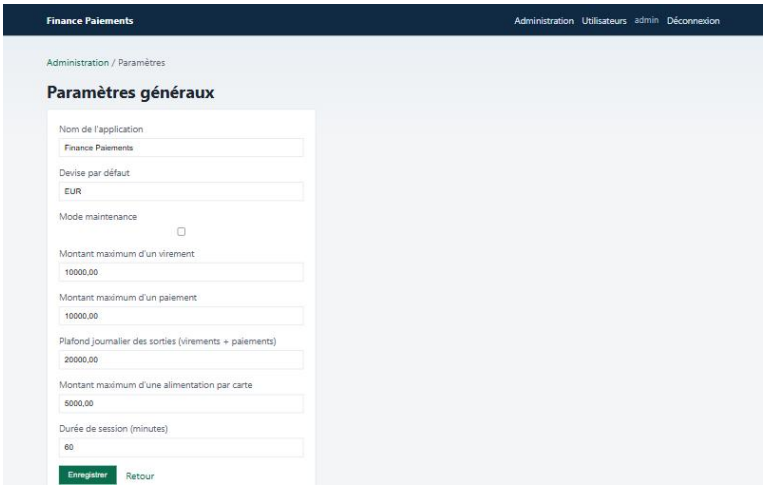


Figure 5.2 – Paramètres : plafonds par opération, journalier et d'alimentation, maintenance

CHAPITRE 6

Tests et validation

6.1 Stratégie

Table 6.1 – Niveaux de test

Niveau	Ce qui est vérifié
Unitaire	Règles métier isolées : solde, plafonds, montants, passerelle de carte.
Intégrité financière	Un échec en cours de transaction n'altère aucun solde.
Intégration	Requête HTTP complète : permissions, codes de retour, rendu des pages et de l'API.
Sécurité	Une attaque par test : accès aux données d'autrui, énumération, double soumission, force brute, CSRF.
Non-régression	Tout défaut corrigé reçoit un test.

6.2 Résultats

Résultats de la campagne de tests (version 1.1.0)

169 tests **169** réussis **86 %** de couverture **2** bases testées

Table 6.2 – Répartition des tests par fichier

Fichier	Tests	Sprint	Domaine
`test_configuration.py`	7	0	Configuration
`test_utilisateurs_wallets.py`	14	1	Utilisateurs, portefeuilles
`test_historique.py`	11	2	Historique
`test_beneficiaires.py`	12	3	Bénéficiaires
`test_virements.py`	15	4	Virements
`test_paiements.py`	13	5	Paiements, remboursements
`test_administration.py`	8	6	Administration
`test_securite.py`	12	7	Security by Design
`test_pilotage.py`	11	8	Pilotage
`test_observabilite.py`	9	9	Observabilité
`test_exports.py`	9	10	Exports
`test_durcissement.py`	48	11	Audit de sécurité
Total	**169**		

6.3 Matrice de traçabilité

Table 6.3 – Matrice de traçabilité exigences / tests (extrait)

Exigence	Tests représentatifs
EF-01 Compte	`test_inscription_web_cree_compte_wallet_et_session` `test_solde_initial_est_zero`
EF-02 Connexion	`test_client_se_connecte_hors_du_site_admin` `test_deconnexion_post_uniquement`
EF-04 Alimentation	`test_alimentation_acceptee_credite_et_trace` `test_alimentation_refusee_ne_credite_pas`
EF-05 Historique	`test_filtres_type_et_statut` `test_historique_automatique`
EF-06 Bénéficiaires	`test_api_ajout_par_email` `test_desactivation_et_reactivation`
EF-07 Virements	`test_virement_valide` `test_transaction_atomique_en_cas_echec_historique`
EF-08 Paiements	`test_paiement_valide` `test_remboursement_montant_excessif`
EF-09 Idempotence	`test_double_soumission_virement_un_seul_debit` `test_api_virement_idempotency_key`
EF-11 Plafonds	`test_plafond_virement` `test_plafond_journalier_cumule`
EF-12 Journal admin.	`test_journal_modification_parametres_et_activation`
EF-13 Pilotage	`test_dashboard_compteurs` `test_statistiques_filtre_utilisateur`
EF-15 Exports	`test_export_tous_types_json` `test_export_csv_neutralise_motif`
ENF-02 Cloisonnement	`test_idor_api_utilisateurs_wallets` `test_acces_interdit_autre_proprietaire`
ENF-03 Annuaire	`test_api_refuse_l_ajout_par_identifiant` `test_formulaire_beneficiaire_ne_liste_pas_les_clients`
ENF-04 Force brute	`test_blocage_apres_echecs` `test_blocage_par_adresse_ip`
ENF-06 Carte	`test_numero_de_carte_jamais_conserve`
ENF-08 Montants	`test_montants_invalides_rejetes`
ENF-12 Production	`test_production_refuse_de_demarrer_sans_cle_secrete`

6.4 Intégration continue

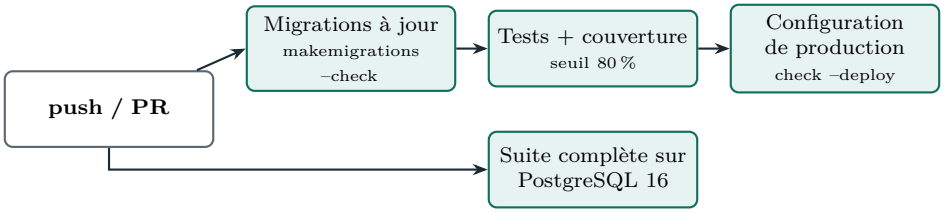


Figure 6.1 – Pipeline d'intégration continue (deux jobs parallèles)

CHAPITRE 7

Audit de sécurité

7.1 Démarche

Le prototype 3 a introduit la *Security by Design* (cloisonnement, plafonds, maintenance, journal d'administration). À la fin du prototype 4, l'auteur a audité son propre code : lecture des services, des vues et des 31 routes d'API, puis parcours manuel complet en tant que client et administrateur. Le constat le plus instructif : deux défauts *bloquants* n'étaient détectés par aucun test, car les tests appelaient directement les services sans jouer le parcours réel d'un client.

7.2 Modèle de menace

Table 7.1 – Actifs, menaces et attaquants

Actif	Menace	Attaquant
Fonds des clients	Double débit, solde négatif, contournement des plafonds	Client, erreur réseau
Annuaire des clients	Collecte des noms et e-mails (hameçonnage ciblé)	Client authentifié
Comptes	Deviner un mot de passe, login CSRF	Attaquant externe
Données d'autrui	Lire un portefeuille ou un historique étranger	Client authentifié
Poste de l'administrateur	Formule exécutée à l'ouverture d'un export	Client malveillant

7.3 Défauts identifiés et corrigés

Table 7.2 – Résultats de l'audit de sécurité (sprint 11)

ID	Gravité	Défaut constaté	Correction
F1	**Critique**	Annuaire énumérable : l'API renvoyait nom et e-mail d'un client à partir d'un identifiant séquentiel ; le formulaire web listait tous les clients.	Ajout de bénéficiaire par e-mail exact, message d'erreur générique.
F2	**Bloquant**	Aucun espace client : seule la connexion `/admin/`, réservée au personnel, existait.	Inscription, connexion, déconnexion (web et API), espace client.
F3	**Bloquant**	Aucun moyen d'approvisionner un portefeuille : l'application était inutilisable.	Alimentation par carte via une passerelle simulée côté serveur.

ID	Gravité	Défaut constaté	Correction
F4	**Élevée**	Double débit sur double clic ou requête rejouée.	Clés d'idempotence et contrainte d'unicité en base.
F5	**Élevée**	Plafond par opération contournable en plusieurs opérations.	Plafond journalier cumulé, contrôlé portefeuille verrouillé.
F6	**Moyenne**	Arrondi silencieux (`0.001` devenait `0.00`) ; erreur 500 sur `1e30`.	Validation stricte des montants.
F7	**Moyenne**	Injection de formules dans les exports CSV.	Neutralisation des cellules à risque.
F8	**Moyenne**	Force brute et login CSRF à l'ouverture de l'espace client.	Blocage temporaire, CSRF imposé, redirections sûres.
F9	**Moyenne**	Adresse IP falsifiable via `X-Forwarded-For`.	En-tête ignoré par défaut.
F10	**Moyenne**	Configuration non sûre par défaut.	Configuration de production dédiée, Docker non-root.
F11	**Qualité**	Fichiers en double interligne, nom de laboratoire visible.	Normalisation, renommage du produit.

7.4 Exemple détaillé : le double débit (F4)

Sans protection, un double clic sur « Exécuter le virement » ou une requête renvoyée après une coupure réseau produisait deux virements. La correction reprend le principe des API de paiement réelles : chaque demande porte une **clé d'idempotence**, et une clé déjà utilisée renvoie l'opération existante au lieu d'en créer une nouvelle.

Listing 7.1 – Virement idempotent : vérification de la clé sous verrou

```
cle = parse_cle_idempotence(cle_idempotence)
if cle is not None:
    # Deux requêtes simultanées portant la même clé sont sérialisées :
    # la seconde retrouve le virement de la première.
    wallet_verrouille = Wallet.objects.select_for_update().filter(
        utilisateur=emetteur).first()
    existant = Virement.objects.filter(
        wallet_emetteur=wallet_verrouille, cle_idempotence=cle).first()
    if existant is not None:
        return existant, False # aucun nouveau débit
```

Listing 7.2 – Test : deux soumissions, un seul débit

```
def test_double_soumission_virement_un_seul_debit(utilisateur, lien_bob):
    premier = executer_virement(emetteur=utilisateur, beneficiaire_id=lien_bob.pk,
                                montant='100.00', cle_idempotence='cle-1')
    second = executer_virement(emetteur=utilisateur, beneficiaire_id=lien_bob.pk,
                               montant='100.00', cle_idempotence='cle-1')
    assert second.pk == premier.pk and second.est_rejeu is True
    assert utilisateur.wallet.solde == Decimal('900.00') # un seul débit
```

CHAPITRE 8

Déploiement

8.1 Environnements

Table 8.1 – Configurations par environnement

Configuration	Usage
`config.settings.local`	Démarrage rapide avec SQLite, sans Docker
`config.settings.development`	Développement avec PostgreSQL (Docker)
`config.settings.test`	Tests automatisés
`config.settings.test_postgres`	Suite de tests sur PostgreSQL
`config.settings.production`	HTTPS, cookies sécurisés, HSTS, WhiteNoise, clé secrète obligatoire

8.2 Mise en production

Le fichier `docker-compose.prod.yml` démarre la base PostgreSQL (volume persistant) et l'application : migrations, collecte des fichiers statiques, puis Gunicorn exécuté par un utilisateur non privilégié. Le chiffrement TLS est assuré en amont par un reverse proxy. Les plafonds, la devise, la durée de session et le mode maintenance se règlent ensuite depuis l'administration.

Listing 8.1 – Lancement local en quelques commandes

```
pip install -r requirements-dev.txt
set DJANGO_SETTINGS_MODULE=config.settings.local
python backend/manage.py migrate
python backend/manage.py creer_comptes_demo
python backend/manage.py runserver 8001
```

CHAPITRE 9

Maintenance et évolutions

9.1 Gestion de version

Le dépôt Git utilise des messages de commit conventionnels et un versionnement sémantique. Le fichier `CHANGELOG.md` retrace chaque version, de la version 1.0.0 (prototypes 1 à 4) à la version 1.1.0 issue de l'audit.

9.2 Limites connues

— La carte bancaire est simulée : la passerelle se remplace par un prestataire réel sans modifier le reste de l'application.

— Pas de double authentification : indispensable avant une mise en service réelle.

— Les compteurs de tentatives de connexion utilisent un cache mémoire, adapté à une instance unique.

9.3 Évolutions envisagées

— Double authentification pour les opérations sensibles.

— Prestataire de paiement réel en mode test.

— Retrait vers un compte bancaire (SEPA).

— Test de bout en bout de chaque parcours client dans la suite automatisée.

CHAPITRE 10

Conclusion

Le projet a mené une plateforme de portefeuille électronique de l'expression des besoins jusqu'à un déploiement de type production, en suivant toutes les phases du cycle de vie du développement logiciel. Le **prototypage évolutif** a permis de sécuriser très tôt le cœur sensible — les mouvements de fonds — puis de l'entourer, prototype après prototype, d'administration, de pilotage et de traçabilité. La méthode **Scrum** a structuré le travail en incréments testés et documentés.

Bilan et rétrospective

Ce qui a bien fonctionné. Le module unique de mouvements de fonds a rendu simples l'ajout de l'idempotence et du plafond journalier ; les 121 tests existants ont confirmé qu'aucune règle financière n'avait régressé.

Ce que je ferais différemment. Écrire dès le départ un test de bout en bout par parcours utilisateur : il aurait révélé immédiatement qu'un client ne pouvait ni se connecter ni approvisionner son portefeuille. Et planifier un audit de sécurité à la fin de chaque prototype.

ANNEXE A

Annexes

A.1 Comptes et cartes de démonstration

Table A.1 – Comptes de démonstration (mode développement uniquement)

Rôle	Identifiant	Mot de passe	Départ
Cliente	`alice`	`Alice-Demo-2026`	1 500 EUR
Client	`bob`	`Bob-Demo-2026`	300 EUR
Administrateur	`admin`	`Admin-Demo-2026`	—

Table A.2 – Cartes de test de la passerelle simulée

Carte	Résultat
`4242 4242 4242 4242`	Acceptée
`4000 0000 0000 0002`	Refusée (fonds insuffisants)
Numéro à clé de Luhn invalide	Rejeté, rien n'est enregistré

A.2 Organisation du dépôt

Table A.3 – Structure du dépôt

Dossier	Contenu
`backend/`	Projet Django, configuration par environnement
`accounts/` … `observabilite/`	Modules métier (modèles, services, vues)
`api/`	API REST
`commun/`	Utilitaires transverses
`frontend/`	Gabarits HTML et CSS
`tests/`	169 tests automatisés
`docker/`	Dockerfile, Compose de développement et de production
`docs/`	Documentation par phase du cycle de vie, ce rapport
`livrables/`	Rapports de sprint et audits